

CSOPESY -- IO Hardware

Device communicates with computer system by sending signals over a cable or even through the air

Device communicates with machine via a connection point - **port** (i.e., serial port)

Devices that share a common set of wires, that connection is called a **bus**

Example: a PCI is a bus used in most computers today.

When device A has a cable that plugs into device B and device B has a cable that plugs into device C and device C plugs into a port on the computer, this arrangement is called a **daisy chain**. A daisy chain usually operates as a bus.

Bus systems are widely used in computer architecture and vary in signaling methods, speed, throughput and connection methods

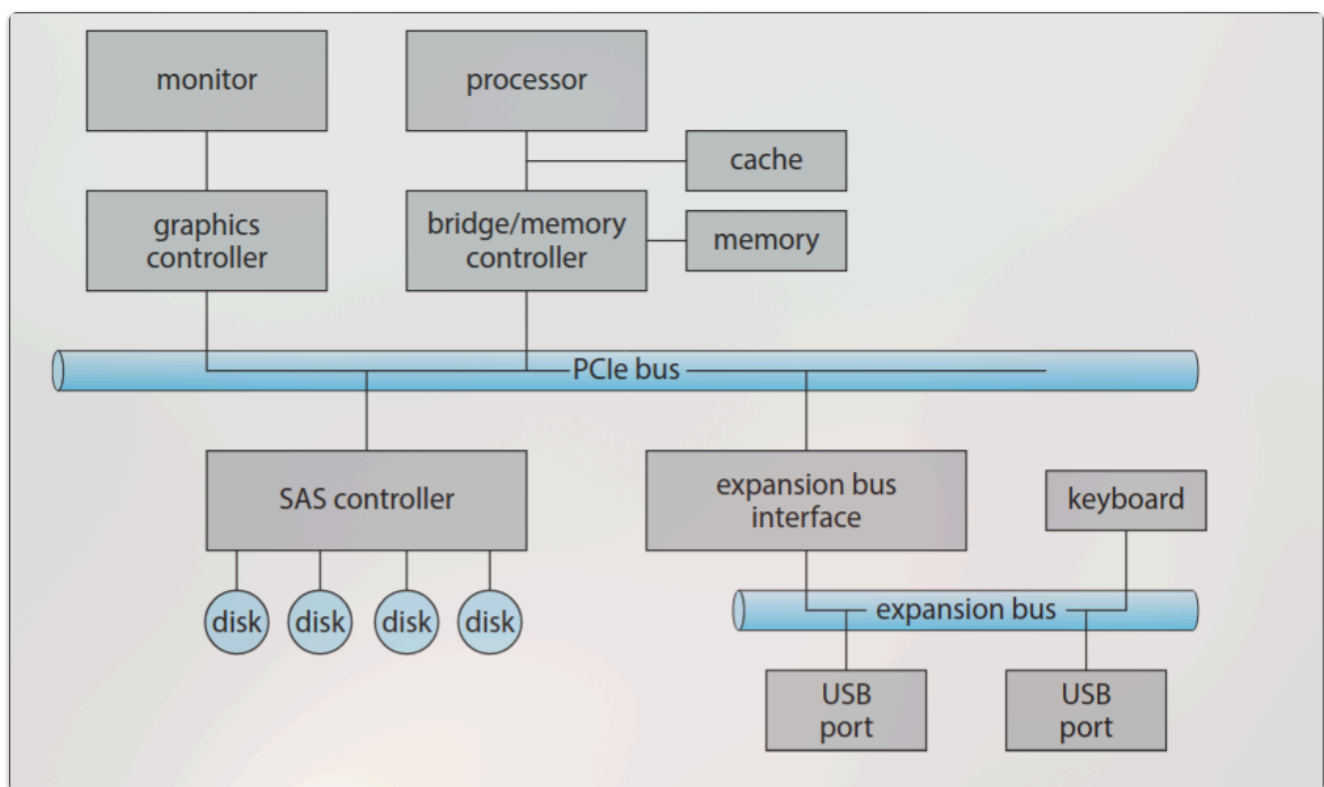


Image 1: Typical bus structure

PCIe links may contain 1, 2, 4, 8, 12, 16 or 32 lanes as signified by an "x" prefix.

Example: PCIe gen3 x8 is that it works with generation 3 of PCIe and uses 8 lanes. Such device has 8 GB per sec

Controller is a collection of electronics that can operate a port, a bus or a device

A **serial-port controller** is a simple device controller. It is a single chip in the computer that controls the signals on the wires of a serial port.

By contrast, a fibr channel (FC) bu scontroller is not simple

Memory-Mapped I/O

instead of having to trigger bus lines to select the proper device and move bits into or out of a device register.

Alternatively, device can support **memory-mapped I/O**. The device-control registers are mapped into address space of the processor. The CPU executes I/O requests using standard data transfer instructions

I/O address range (hexadecimal)	device
000–00F	DMA controller
020–021	interrupt controller
040–043	timer
200–20F	game controller
2F8–2FF	serial port (secondary)
320–32F	hard-disk controller
378–37F	parallel port
3D0–3DF	graphics controller
3F0–3F7	diskette-drive controller
3F8–3FF	serial port (primary)

Image 2: Device I/O port locations on PCs (not entire)

I/O device control typically consists of four registers called the status, control, data-in, data-out registers

- **data-in register** is read by the host to get input
- **data-out register** written by the host to send output
- **status register** contains bits that can be read by the host. Indicate states, such as whether the current command has been completed
- **control register** can be written by the host to start a command or to change mode of a device

The data registers are typically 1 to 4 bytes in size.

Polling

Complete protocol for interaction between host and controller

For each byte of I/O:

1. The host repeatedly reads the busy bit until that bit becomes clear.
2. The host sets the write bit in the command register and writes a byte into the data-out register.
3. The host sets the command-ready bit.
4. When the controller notices that the command-ready bit is set, it sets the busy bit.
5. The controller reads the command register and sees the write command. It reads the data-out register to get the byte and does the I/O to the device.
6. The controller clears the command-ready bit, clears the error bit in the status register to indicate that the device I/O succeeded, and clears the busy bit to indicate that it is finished.

Step 1 is busy wait cycle to wait for I/O from device

- reasonable if device is fast but inefficient if device slow
- CPU switches to other tasks but if i miss a cycle data is overwritten.

Interrupts

Imagine polling rate of a mouse, thats how fast something is being asked by the CPU if something changed in the input

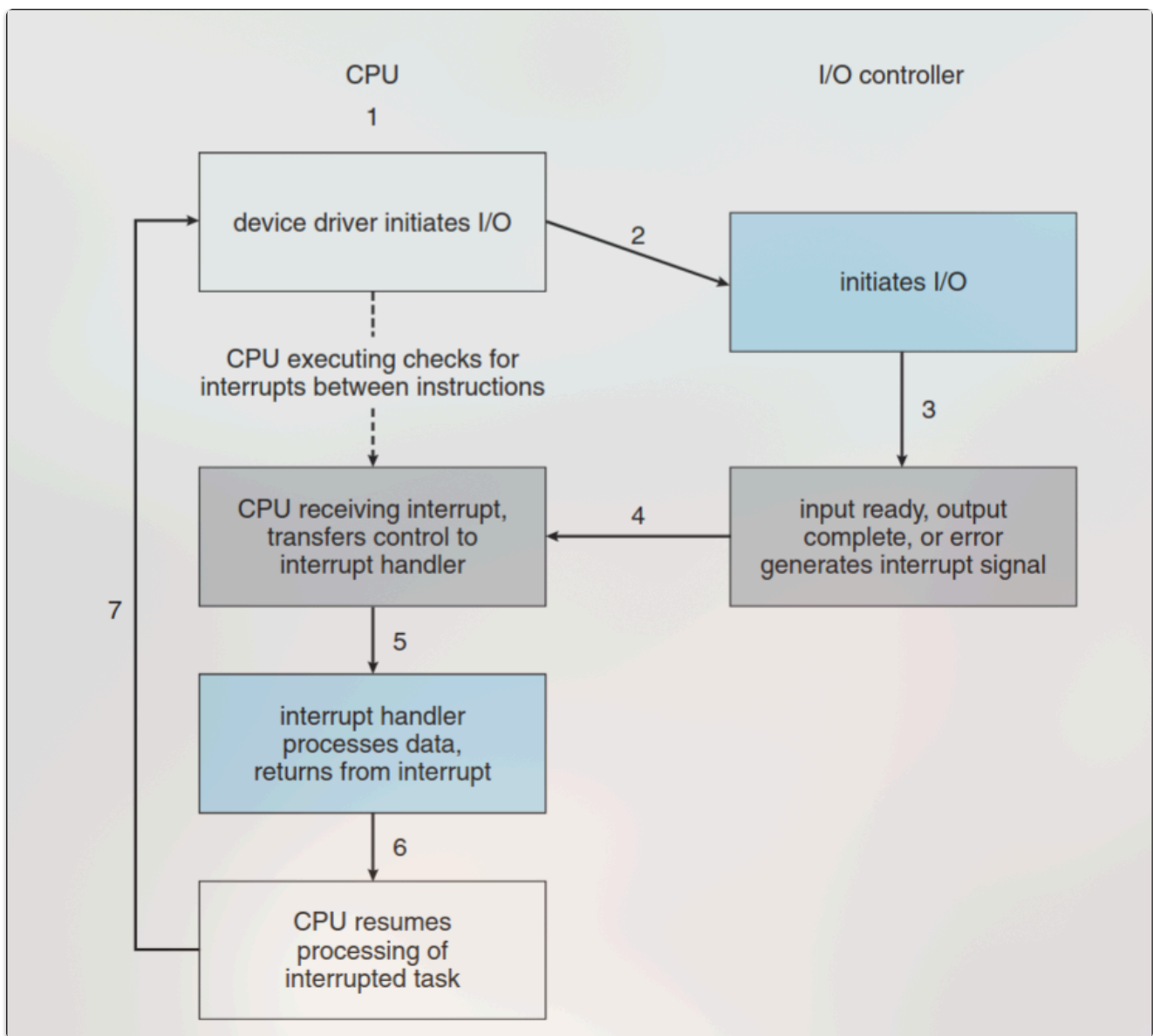


Image 4: Interrupt driven I/O Cycle

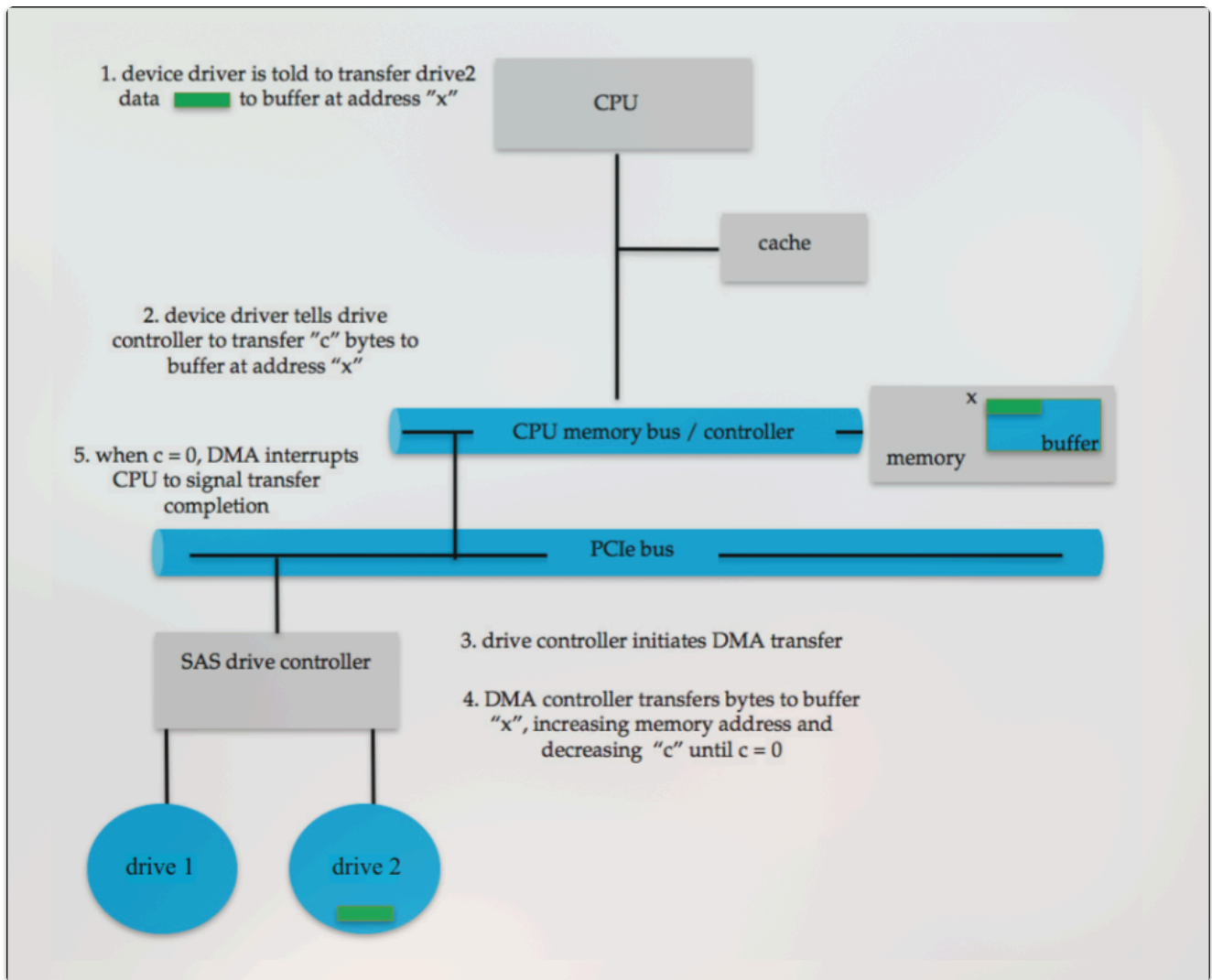
- CPU interrupt request line triggered by I/O device
- interrupt handler receives interrupts
- interrupt vector
 - dispatch interrupt to correct handler
 - context switch at start and end
 - based on priority
 - some nonmaskable
 - interrupt chaining if more than one device at same interrupts number
- used for exceptions

Basic interrupt mechanism enables the CPU to respond to an asynch event, as when a device controller becomes ready for service

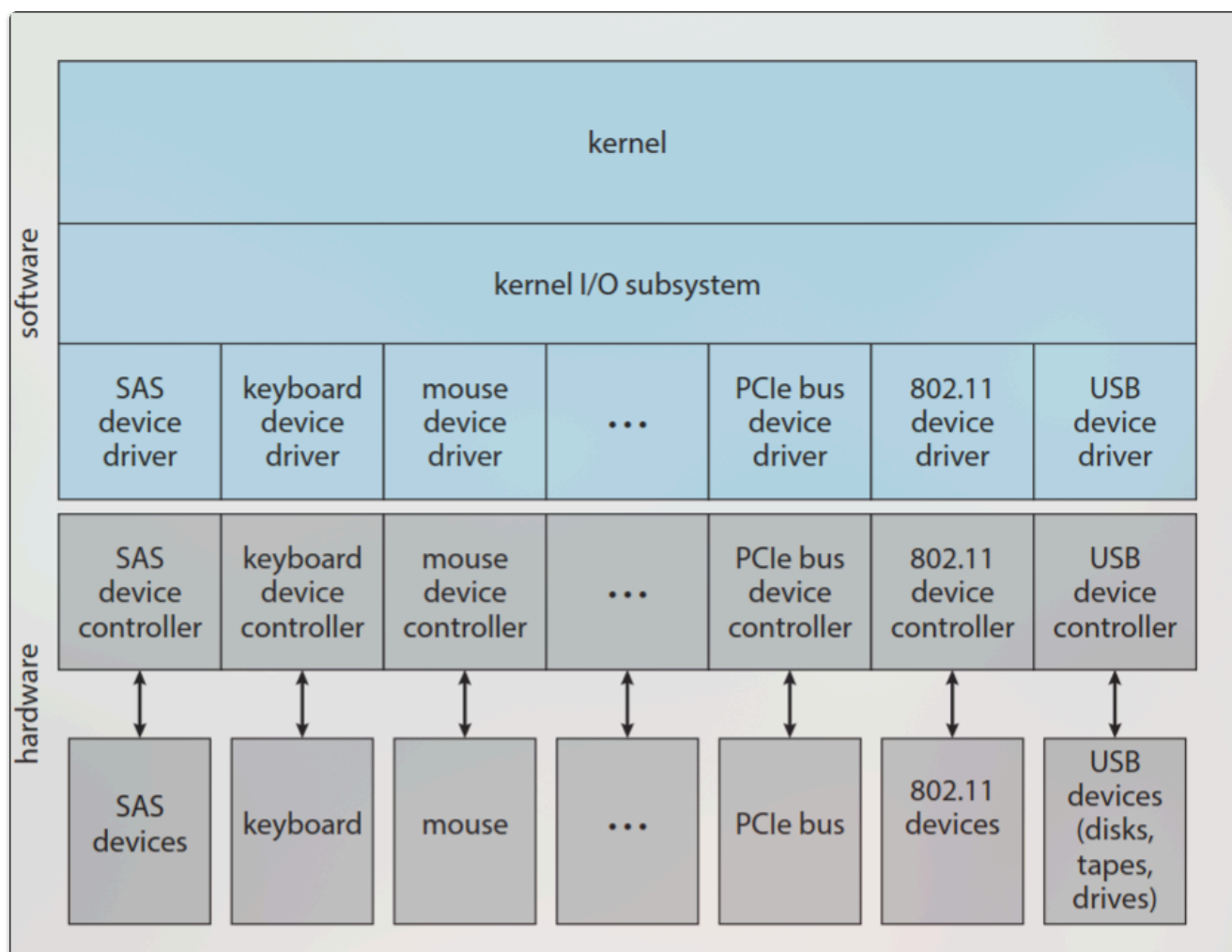
1. We need the ability to defer interrupt handling during critical processing.
2. We need an efficient way to dispatch to the proper interrupt handler for a device without first polling all the devices to see which one raised the interrupt.
3. We need multilevel interrupts, so that the operating system can distinguish between high- and low-priority interrupts and can respond with the appropriate degree of urgency when there are multiple concurrent interrupts.
4. We need a way for an instruction to get the operating system's attention directly (separately from I/O requests), for activities such as page faults and errors such as division by zero. As we shall see, this task is accomplished by "traps."

Direct Memory Access

- used to avoid programmed I/O (one byte at a time) for large data movement
- requires DMA controller
- bypasses CPU to transfer data directly between I.O device and memory
- OS writes DMA command block into memory
- we use this because it makes CPU more efficient. it makes the usage of computing power less as DMA takes away from the CPU the power it needs for I/O
- con is that its a piece of hardware



Application I/O Interface



Kernel I/O Structure

Each general kind of the differences in I/O devices is accessed through a standardized set of functions called an **interface**.

The differences are encapsulated in kernel modules called **device drivers** that are internally custom-tailored to specific devices but that export one of the standard interfaces.

Refer to Kernel I/O structure.

The purpose of the device driver layer is to hide the differences among device controllers from the I/O subsystem of the kernel.

Making the I/O subsystem independent of the hardware simplifies job of OS dev

aspect	variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	tape keyboard
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read–write	CD-ROM graphics controller disk

Characteristics of I/O devices

Device characteristics:

1. Character-stream or block

- a character-stream device transfers bytes one by one whereas a block device transfer a block of bytes as a unit

2. Sequential or random access

- sequential device transfer data in a fixed order determined by device, random-access device can instruct device to seek to any of available data storage locations

3. Synchronous or Asynchronous

- synchronous device performs data transfers with predictable response times, in coordination with other aspects of the system. An asynch device exhibits irregular or unpredictable response times not coordinated with other computer events

4. Sharable or dedicated

- sharable device can be used concurrently by several processes or threads; dedicated device cannot

5. Speed of operation

- device speeds range from few bytes per second to gigabytes per second

6. Read-write, read only, write once

- some devices perform both input and output
- but others support only one data transfer direction.
- some allow data to be modified after write but others can be written only once and are read-only thereafter

For purpose of application access, many of these are hidden by the OS.

Resulting styles of device access have been found to be useful and broadly applicable. Although exact system calls may differ across operating systems.

Most OS also have an escape or backdoor that transparently passes arbitrary commands from application to a device driver.

- this `ioctl()` syscall enables application any functionality that can be implemented by any device driver without need to invest a new system call

Network Devices

- varying enough from block and character to have own interface
- Linux, Unix, Windows, and many others include socket interface
- separates network protocol from network operation
- includes `select()` functionality – returns info about which sockets have a packet waiting; eliminates polling
- approaches vary widely (pipes, FIFOs, streams, queues, mailboxes)

Clocks and Timers

- provide current time, elapsed time, timer
- normal resolution about 1/60 second
- some systems provide higher-resolution timers
- programmable interval timer used for timings, periodic interrupts
- `ioctl()` (on UNIX) covers odd aspects of I/O such as clocks and timers

Nonblocking and Asynchronous I/O

- Blocking: process suspended until I/O completed
- easy to use and understand
- insufficient for some needs
- Nonblocking: I/O call returns as much available
- user interface, data copy (buffered I/O)
- implemented via multi-threading
- returns quickly with count of bytes or written
- `select()` to find if data ready then `read()` or `write()` to transfer
- Asynchronous: process runs while I/O executes
- difficult to use
- I/O subsystem signals process when I/O completed

Vectored I/O

- allows one system call to perform multiple I/O operations

- single procedure call sequentially reads data from multiple buffers or reads data from a data stream and writes it to multiple buffers
- scatter-gather method better than multiple individual I/O calls
- decreases context switching and system call overhead
- some versions provide atomicity

Kernel I/O Subsystem

- Scheduling
- some I/O request ordering via per-device queue
- some OSs try fairness
- some implement Quality of Services (i.e. IPQOS)
- Buffering: store data in memory while transferring between devices
- Caching: faster device holding copy of data
- Spooling: hold output for a device
- Device Reservation: provides exclusive access to a device

Power Management

- not strictly domain of I/O, but much is I/O related
- computers and devices use electricity, generate heat, frequently require cooling
- OSes can help manage and improve use
- Ex: wake locks, power collapse

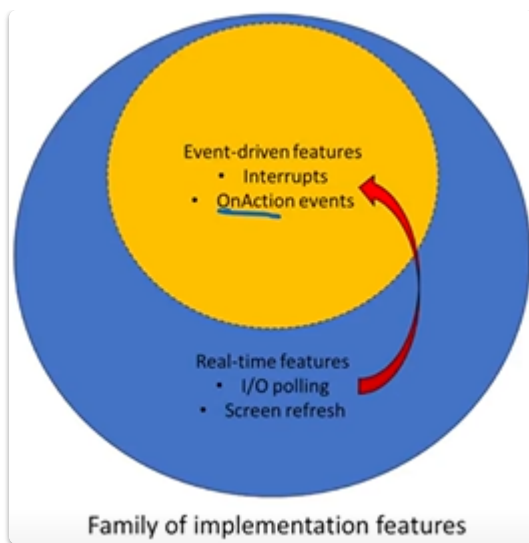
Keyboard Polling

- means to support keyboard input, both real-time and event-driven
- Real-time: user can always type characters, and this displays on-screen immediately (e.g. word processors); OS-driven (e.g. Windows UI)
- Event-driven: wait for an event, such as an enter command, and then perform an associated operation; user applications-driven (e.g. Java GUI)
- all are derived from real-time applications; still require functionality that needs to run in real time
- Ex: web applications, mobile applications, networking applications
- applications can "switch" real-time mode to event-drive mode and vice versa, and they can also run together

Polling in OS

- must be done in real-time rather than event-driven
- if done with `std::cin` for instance, it will indefinitely wait for input
- without using a background worker, emulator will stall and not do anything

Family of Implementation Features



- Ex: a background in hardware-level I/O systems

Interrupt Mechanism

- present in all hardware
- in an interrupt-driven system, CPU is interrupted when a key is pressed
- OS or a low-level handler responds to the interrupt and takes appropriate action:
- a system API call could be exposed where events can be handled by the developer (e.g. `onKeyDown()`, `onKeyUp()`)
- more event-driven and can be more efficient as CPU is not constantly checking for input
- it first interrupts ("pauses") to poll for a response, then refreshes to draw/update